Cursor — compiled path

> Compiled from `/home/dnikolic/Documents/Learning/Plan ucenja/paths/Cursor`. Canonical sources stay in place.

Phase — 01-cursor-foundations-and-mental-model

Directory: `01-cursor-foundations-and-mental-model/`

Source: `01-cursor-foundations-and-mental-model/01-scope-ai-pair-programming-trust-boundaries-calibration-and-failure-classes.md`

Unit 1 — Scope: Cursor foundations — deterministic engineering with probabilistic copilots

Mindset shift: treat the assistant as a **high-bandwidth collaborator** bounded by probabilistic behaviour — not magic, not an oracle.

Learning outcomes

- **Probabilistic vs deterministic tooling**: compilers tests lockfiles → deterministic bounds; completion → distribution over plausible edits.
- **AI pair-programming ethic**: humans keep **architecture + ethics + escalation** accountability; assistants accelerate exploration and scaffolding.
- **Ownership boundaries**: you own merges, regressions, security, and stakeholder truth — not “model said OK”.
- **Trust model tiers**: verified facts (syntax, searched code) vs synthesized explanations vs speculative refactor plans — escalate verification effort accordingly.
- **Verification reflex**: bias toward **minimal reproduction**: diff + test + runnable proof before belief.
- **Failure classes taxonomy** (baseline vocabulary):
- **Hallucinated symbol / path / API**.
- **Overconfident refactor** violating implicit constraints.
- **Context starvation** / wrong file grounding.
- **Instruction ambiguity** collapsing into wrong task.
- **Completion bias** patching symptoms not causes.
- **Format-only compliance** (“looks merged”) without semantic integration.
- **Confidence calibration prompts**: practise describing uncertainty tiers out loud (“I’m speculating…” vs “I measured…”).
- **Ambiguity ownership**: if specs are muddy, escalate to humans/specs — avoid laundering ambiguity via AI guesses.
- **Workflow discipline hooks**: repeatable prompt templates (`goal → constraints → forbidden edits → verification steps`).
- **Engineering humility**: optimise for reversible steps, checkpoints, rollback-friendly branches.

Practice spine across this trace: reuse **one flagship repo** slice (Symfony + Go + infra bits you maintain) whenever exercises ask for artefacts—keep boundaries consistent.

Source: `01-cursor-foundations-and-mental-model/02-labs-trust-matrix-failure-triage-mini-audit-and-guard-phrases.md`

Unit 2 — Labs: classify — label — escalate

Drill A — Trust tier table

Paste a realistic assistant answer (past chat or synthetic). Annotate sentences with **FACT / DERIVED / SPECULATIVE** using evidence citations (paths, docs, profiler output).

Drill B — Failure class bingo

Maintain a personal **five-incident corpus** spanning at least four failure classes from the scope taxonomy. Write one-line RCA + corrective habit per incident.

Drill C — Guard-phrase playbook

Produce **five** reusable guard phrases rejecting unsafe assistance (e.g. secret sprawl, mass delete without plan, speculative schema migration).

Drill D — Ambiguity escalation template

Craft a terse template you would send upstream PM/architect listing **ambiguous requirement → blocker risk → clarification question**.

Deliverable checklist

Minimum artefact bundle for this phase:

```
trust-tier-example.md
failure-classes.md (table)
ambiguous-escalation-template.md
```

Interview stance: articulate **deterministic tooling vs probabilistic copilot** plus how you **prevent silent authority transfer** to automation.

Phase — 02-specification-driven-engineering

Directory: `02-specification-driven-engineering/`

Source: `02-specification-driven-engineering/01-scope-gsd-sdd-owned-acceptance-nfrs-and-spec-drift.md`

Unit 1 — Scope: specification-driven engineering (SDD mindset)

Mindset shift: **spec-first** artefacts — machine- or human-auditable — precede exploratory coding; ambiguity is surfaced early, never sidestepped with generation.

Learning outcomes

- **Goal Specification Development / SDD hybrids**: unify goal statements → constraints → observable acceptance probes.
- **Executable specification ethos**: behaviours expressible as verifiable predicates (tests, linters, contract checks) even when not literal DSL.
- **Ownership lattice** explicit for:
- **Acceptance** (business-visible proof).
- **Non-functional constraints** (latency envelopes, tenancy, auditing).
- **Definition of Done** bridging release + rollback posture.
- **Requirement completeness** sanity (negative paths, quotas, abusive usage).
- **Failure ownership** when spec contradictions emerge mid-build.
- **Specification hierarchy navigation**: roadmap → milestone → epic → story → invariant → task — keep mapping consistent.
- **Specification drift signals**: creeping scope, silent relaxations (“quick hack”), divergence between docs/tests/code.
- **Implementation consistency checkpoints**: refactor cannot silently orphan earlier acceptance stories.
- **Boundary ownership**: module boundaries coincide with behavioural contracts (DDD-friendly without mandating jargon).
- **Validation loops**: reviewer + tooling + exploratory manual scenario triangulation.

Linkage forward: richer context layering appears in `03-*`; governance gates in `06-*`; runtime enforcement in `16-*` (specification-ish checks at integration edges).

Source: `02-specification-driven-engineering/02-labs-spec-sheet-acceptance-tests-and-drift-watch.md`

Unit 2 — Labs: spec artefacts that bite back

Pick a bounded feature stub (payments retry idempotency, coupon validation, CQRS-ish read-model refresh — choose one coherent slice).

Lab 1 — One-page behavioural spec

Write ≤400 words capturing **Actors → Preconditions → Main flow → Alternate / failure flows → Acceptance signals → Non-goals**.

Lab 2 — Acceptance table

Enumerate **given / when / then** scenarios including **minimum one malicious / abuse case**.

Lab 3 — NFR appendix

Bullet **measurable** envelopes (latency, throughput budget, tenancy isolation, auditing). Flag anything unmeasurable → needs redesign.

Lab 4 — Drift radar

Maintain a `DRIFT-WATCH.md` diff log across two pseudo “sprints” where you deliberately mutate scope—record detection mechanism (test failure, reviewer comment).

Deliverable expectation

Produce a **minimal test skeleton** aligning with acceptance rows (pseudo-code tolerated if toolchain mismatch, but predicates must compile mentally).

Phase — 03-context-engineering-and-layering

Directory: `03-context-engineering-and-layering/`

Source: `03-context-engineering-and-layering/01-scope-context-architecture-retrieval-session-and-compression-tradeoffs.md`

Unit 1 — Scope: context engineering — context *is* design

Mindset shift: optimise **instruction + corroborating evidence bundles** deliberately — not shovel entire repos into chats.

Learning outcomes

- **Context ownership**: who constructs, refreshes, and retires ephemeral instruction state.
- **Prioritisation heuristics**:
- Symptom → nearest failing test → callee stack → owning module.
- Feature → contract first → adapters second → internals last.
- **Layering blueprint**:
- **Goal / invariant**
- **Pinned evidence** (file paths, excerpts, profiler snapshot)
- **Operating constraints**
- **Forbidden moves**
- **Verification plan**
- **Rollback / checkpoints**
- **Compression discipline**: summarise large regions with **anchors** (symbols, hashes) referencing repo truth — forbid narrative-only invention.
- **Retrieval mindset**: treat search tools as stochastic — cross-check grounding (open file excerpt > memory).
- **Instruction hierarchy**: precedence when layers conflict (“security constraints override ergonomics refactor”).
- **Context budgeting**: approximate token-ish weighting — escalate summarisation triggers.
- **Memory / persistent artefacts**: Cursor rules, SKILL files, AGENTS stubs — versioning + staleness auditing.
- **Session ownership**: when to fork new chat vs continue polluted thread.
- **Dependency context hygiene**: pinning library versions constraints for assistant suggestions referencing APIs.
- **Architecture preservation snippets**: capsules of ADR excerpts + diagram snapshot lines.

Adjacent work: overlaps `04-*` (prompt shapes) `19-*` (cost) `12-*` (workflow minimalism anti-patterns).

Source:
`03-context-engineering-and-layering/02-labs-context-budget-mini-pack-and-grounding-protocol.md`

Unit 2 — Labs: build a repeatable context capsule

Pick a moderately complex regression (duplicate request storm, flaky integration, migration half-applied hypothetical).

Lab A — Two-tier capsule templates

Produce **SHORT** (~25 lines equivalent) vs **FULL** capsules following the layering blueprint — measure subjective “mental load proxy” (#files referenced, redundancy).

Lab B — Retrieval failure injection

Draft instructions where naive search retrieves **wrong sibling module** document how you preempt with discriminators (paths, namespaces, bounded search roots).

Lab C — Compression stress

Given a 250-line excerpt, summarise to ≤40 lines preserving **identifiers + behavioural guarantees** only.

Lab D — Stale SKILL / Rules audit

Enumerate current workspace guidance files; classify **AUTHORITATIVE** / **HISTORICAL** / **CONFLICTING** propose merge plan.

Deliverable artefact bundle: `context-capsules/*.md` with both tiers + rationale note tying budget decision to reviewer burden.

Phase — 04-prompt-engineering-best-practices

Directory: `04-prompt-engineering-best-practices/`

Source: `04-prompt-engineering-best-practices/01-scope-constrained-decomposed-reviewed-and-validated-instructions.md`

Unit 1 — Scope: prompt engineering — engineered instructions, not vibes

Mindset shift: prompts are **specifications for statistical programs** — structure beats cleverness.

Learning outcomes

- **Constraint prompting**: explicit forbids (“do not mutate migrations”, “preserve public signatures”).
- **Architecture prompting**: package boundaries / bounded context bullets first.
- **Decomposition prompting**: staged micro-tasks minimizing cross-file thrash risk.
- **Specification prompting**: restate acceptance as machine-checkable where possible.
- **Retrieval prompting**: define search anchors not vague “investigate slowdown”.
- **Review prompting**: adversarial reviewer persona with checklist (security regressions diff focus).
- **Repair prompting**: post-failure deltas—what hypothesis failed vs next measurement.
- **Migration / refactor prompting**: compatibility invariants enumerated first.
- **Validation prompting**: require evidence blocks (snippet + command + expectation).
- **Anti-hallucination hygiene**: forbid invention of unseen symbols; cite or mark UNKNOWN.
- **Chain ownership**: who approves chaining multi-step autonomy vs human gates each hop.
- **Output shaping**: response format enforcing diff hunks vs narrative-only drift.
- **Context optimisation trade-offs**: iterative narrowing vs risking omitted evidence.
- **Failure prompting hooks**: escalate when contradictory instructions detected.
- **Approval / rollback meta prompts**: scripted language for cancelling unsafe partial application.

Linkage: amplifies `03-*` layering; dovetails `06-*` governance language for automatic vs manual chains.

Source: `04-prompt-engineering-best-practices/02-labs-prompt-pattern-catalogue-and-adversarial-review.md`

Unit 2 — Labs: prompt patterns library & failure matrix

Maintain `prompt-kit/` (markdown snippets you can paste/adapt—not blind copy).

Tasks

1. **Constraint refactor prompt** rewriting a sloppy example into forbids-first structure.
2. **Review persona prompt** insisting on SECURITY + BREAKING CHANGE scan.
3. **Diagnostic prompt** forbidding speculative stack traces absent reproduction.
4. **Migration prompt** requiring expand/contract sequencing language.
5. **Validation prompt** demanding before/after command outputs as evidence.
6. **Self-contradiction trap**: draft prompt with internal conflict → document assistant mis-optimisation → fixed version.
7. **Output shape A/B**: same task with freeform vs structured diff template—note review velocity delta subjectively.

Deliverable: **catalogue table** — pattern name | when to use | anti-pattern it prevents | verification hook.

Phase — 05-code-generation-ownership

Directory: `05-code-generation-ownership/`

Source: `05-code-generation-ownership/01-scope-scaffolding-diffs-migrations-and-debt-ownership.md`

Unit 1 — Scope: code generation ownership — safe acceleration

Mindset shift: generation is **provisional** until integrated, tested, reviewed under human architectural authority.

Learning outcomes

- **Scaffolding ownership**: templates must align with project conventions (lint, fmt, module layout).
- **Incremental generation**: vertical slices > horizontal carpet refactors.
- **Implementation ownership**: merge author understands every line or has explicit delegation + follow-up learning plan.
- **Architecture & constraint preservation**: guard rails for public API stability, error semantics, logging shape.
- **Compatibility ownership**: semver / DB migration pairing awareness.
- **Migration ownership**: dual-write / backfill / validation windows articulated before code drops.
- **Diff ownership**: readable hunks, logical commit boundaries, bisect-friendly history.
- **Rollback ownership**: feature flags, toggles, revert strategy per risky commit.
- **Boundary preservation**: no silent cross-module leakage “because AI suggested import”.
- **Dependency ownership**: upgrade paths vetted (license, breaking notes, supply chain).
- **Technical debt ledger**: track expedient AI patches → scheduled paydown.
- **Refactor ownership**: behavioural equivalence arguments (tests + invariants).
- **Regression ownership**: expand tests *before* wide automated edits when feasible.
- **Consistency**: generated comment tone / docblocks must not fight house style.

Cross-links: verification cadence `07-*`, governance `06-*`, failure patterns `08-*`.

Source: `05-code-generation-ownership/02-labs-preflight-staging-debt-ledger-and-rollback-rehearsal.md`

Unit 2 — Labs: generation preflight + integration audit

Select a **non-trivial** but bounded task (e.g. add idempotency key handling to a command handler stub).

Lab 1 — Preflight contract

List **invariants** + **forbidden edits** + **verification commands** before any generation.

Lab 2 — Staged plan

Break into **≤3** integration-safe steps with explicit checkpoint after each (tests green).

Lab 3 — Diff narrative

After execution (human or assisted), author **commit message story** mapping each hunk to invariant.

Lab 4 — Debt note

If expedient shortcut taken, file **`DEBT.md` entry** with trigger condition for repayment.

Lab 5 — Rollback rehearsal

Describe exact revert / flag flip / DB forward-fix path if deployment misbehaves.

Success metric: another engineer (or future you) can **review without re-running model** and still trust intent.

Phase — 06-ai-governance-and-safe-adoption-engineering

Directory: `06-ai-governance-and-safe-adoption-engineering/`

Source: `06-ai-governance-and-safe-adoption-engineering/01-scope-approval-escalation-audit-and-trust-boundary-policies.md`

Unit 1 — Scope: AI governance engineering — policy becomes product

Mindset shift: assistant usage is an **operational surface** — approvals, audit, escalation, least privilege.

Learning outcomes

- **Approval workflows**: when human sign-off mandatory (schema, auth, billing, PII).
- **Human approval gates** vs automated checks—matrix by risk class.
- **Escalation ownership**: ambiguous security → security owner, not smartest IC guess.
- **Fallback ownership**: degrade to manual authoring / smaller model / narrower tool window.
- **Audit ownership**: what gets logged (prompt ids, approvals, artefacts) respecting privacy/reg constraints.
- **Operational policy**: permissible data classes in AI context windows.
- **Compliance hooks**: DPIA-lite thinking, residency awareness (high-level, verify legally).
- **Rollback / incident hooks**: revoke tokens, purge cached context exposures.
- **Governance RACI overlay** for organisational AI adoption.
- **Trust boundary ownership**: where AI suggestions **must not cross** CI secrets, prod creds paths.
- **Safe generation posture**: forbid secret pasting patterns, red-team self prompts.
- **Permission ownership**: aligning repo privileges with assistant capabilities / MCP tooling.
- **Organisational AI rules layering**: Cursor rules vs central policy vs team conventions—conflict precedence.

Feeds production alignment `20-*`, tooling `14-*`, verification `07-*`.

Source: `06-ai-governance-and-safe-adoption-engineering/02-labs-mini-policy-matrix-and-incident-excerpt-drill.md`

Unit 2 — Labs: design a miniature AI usage policy matrix

Assume a product org (~40 engineers). Produce:

Section A — Risk classes

Enumerate **≤8** assisting-risk classes with example scenarios (billing, infra, datastore, UX copy, OSS license).

Section B — Approval matrix table

Risk class | Automated OK? | Mandatory reviewers | Artefacts | Logging | Fallback | SLA

Section C — Incident runbook excerpt

Leak suspicion / bad merge / MCP tool misuse — **5-step** playbook.

Section D — Forbidden context list

Concrete examples **never** pasted or indexed (pseudo tokens acceptable).

Deliverable reviewer question: Where would bureaucratic friction **prevent real velocity** vs where friction **saves the company**? Document two adjustments.

Phase — 07-verification-engineering

Directory: `07-verification-engineering/`

Source: `07-verification-engineering/01-scope-layered-evidence-contracts-consistency-and-drift.md`

Unit 1 — Scope: verification engineering — evidence beats narration

Mindset shift: model output is **untrusted until validated** via layered checks—not single glance review.

Learning outcomes

- **Verification ownership tiers**: unit, contract, integration, smoke, exploratory.
- **Acceptance verification** tying human-visible behaviour to artefacts.
- **Specification verification**: traceability rows ↔ tests/checks.
- **Implementation verification**: diff aligns with invariant list.
- **Constraint validation hooks**: linter + policy-as-code snippets.
- **Output validation**: generated files compile, format, lint, typed if applicable.
- **Regression validation**: flaky signal triage—not ignored because “AI authored”.
- **Architecture verification**: module dependency rules / import boundaries.
- **Consistency verification**: log field stability, telemetry schema coherence.
- **Trust verification discipline**: rerun failing command after assistant claims fix.
- **Drift verification**: detect silent behaviour change lacking test update.
- **Correctness narratives**: inversion—how would we know we were silently wrong?
- **Benchmark-verified claims** where performance touted.
- **Repair ownership**: if verification fails → structured diagnostic chain (hypothesis backlog).

Amplifies `11-*` testing depth; dovetails `18-*` evaluation metrics mindset.

Source: `07-verification-engineering/02-labs-verification-ladders-falsification-and-drift-detectors.md`

Unit 2 — Labs: verification ladders & falsification drills

Pick a flaky story: assistant “fixed” flake by widening timeout (anti-pattern scenario).

Task 1 — Ladder blueprint

Enumerate minimum checks before merge for **THREE** severity classes (`cosmetic util`, `stateful svc`, `data migration glue`).

Task 2 — Falsification test

Craft a deliberate **counterexample** proving old fix insufficient—encode as failing test premise.

Task 3 — Trust breaker protocol

Five-item checklist forcing **instrumentation first**—no blind retry loops.

Task 4 — Drift detector stub

Ideate static rule or CI script catching **silent interface drift** across packages (pseudo-config ok).

Interview angle: articulate difference between **testing** coverage theatre vs **verification** signalling decision quality under AI acceleration.

Phase — 08-failure-engineering-and-recovery

Directory: `08-failure-engineering-and-recovery/`

Source: `08-failure-engineering-and-recovery/01-scope-classifying-ai-related-failures-repair-workflows-and-escalation.md`

Unit 1 — Scope: failure engineering — operationalise probabilistic breakage

Mindset shift: classify AI-assisted mishaps systematically **before blame** → repair pipeline.

Learning outcomes

- Failure archetypes aligning with foundational taxonomy plus:
- **Ambiguity collapse**
- **Specification contradiction**
- **Constraint violation**
- **Context starvation**
- **Hallucinated dependency**
- **Implementation drift**
- **Ownership drift**
- **Token / window pressure artefacts**
- **Retry amplification loops**
- **Tool / dependency outages**
- **Recovery ownership**: restore known-good state baseline.
- **Classification rubric**: triage urgency + blast radius quickly.
- **Repair workflow canonical steps**: STOP → Snapshot evidence → Isolate minimal repro → Decide human vs autonomous attempt → Instrument → Retry bounded → Escalate.
- **Escalation workflow**: triggers (security ambiguous, systemic perf regression, unresolved contradiction).
- **Post-repair enrichment**: tighten rule / prompt capsule / SKILL / checklist.

Feeds orchestration narratives `13-*`, tooling degradation `14-*`.

Source: `08-failure-engineering-and-recovery/02-labs-failure-scenario-cards-and-repair-game.md`

Unit 2 — Labs: failure cards + rehearsal tabletop

Produce **six failure scenario cards**, each structured:

Situation • First symptom • Probable misclassification • Measurement step • Correct class • Owned repair • Preventive guard

Include at least: **ambiguous spec drift**, **context truncation mid-refactor**, **hallucinated import**, **silent logging shape break**, **unbounded retry churn**, **MCP tooling partial outage**.

Facilitate a **solo tabletop**:

1. Draw two cards blindly (random).
2. Timebox **7 minutes**: execute repair workflow verbally (no tooling).
3. Write retro: which guardrail you will add (**rule / SKILL / CI / reviewer checklist**).

Bonus: correlate each card trigger to at least **one proactive metric** observable in `20-*` production chapter.

Phase — 09-repository-understanding-and-architecture-discovery

Directory: `09-repository-understanding-and-architecture-discovery/`

Source: `09-repository-understanding-and-architecture-discovery/01-scope-indexing-mapping-ddd-cqrs-hotspots-and-legacy-archaeology.md`

Unit 1 — Scope: repository understanding systems — map before motion

Mindset shift: navigation + decomposition precedes refactor; comprehension is deliberately constructed.

Learning outcomes

- **Indexing mental model**: what's cheap vs costly to traverse; hotspots vs periphery.
- **Ownership mapping artefacts**: TEAM / OWNERS hints, bounded contexts overlays.
- **Architecture extraction drills**: peeling adapter vs domain vs infra layers pragmatically when docs stale.
- **Dependency ownership graphs**: cyclic smell triage—noticing fan-in complexity before AI suggests "clever split".
- **CQRS intuition**: classify command vs query paths aligning files.
- **DDD pragmatic lens**: aggregates as consistency islands—avoid careless cross-invariant edits.
- **Package / module cohesion metrics** (conceptual—even if heuristic line counts initially).
- **Coupling hotspots**: brittle shared DTO crates, leaking infrastructure into domain folders.
- **Bounded context leakage detection** questions.
- **Ownership hierarchy overlays** vs flat directory listings.
- **Legacy understanding playbook**: archaeology without rewrite hubris driven by completions.

Feeds refactor chapter `10-*`, orchestration decomposition `13-*`.

Source: `09-repository-understanding-and-architecture-discovery/02-labs-repo-cartography-hotspots-diagram-forbidden-move-list.md`

Unit 2 — Labs: repository cartography sprint

Goal: ship `REPO-NOTES.md` (living document) answering:

Sections (mandatory headings)

A. Big picture runtime surfaces

Enumerate entrypoints / workers / consoles / cron / HTTP handlers (~10 bullets max).

B. Domain slices & boundaries

Bulleted inferred contexts + **risky couplings**.

C. Ownership guess map

Stakeholder / team guesses + unknowns flagged.

D. Complexity budget hotspots

Five candidates for future modularisation—not solutions yet.

E. Forbidden assistant moves

Concrete anti-patterns (e.g., “bulk rename without migration plan”) tied to hotspots.

Include **diagram**: simple boxes + directional edges (PlantUML tolerated).

Timebox extraction to **≤90 human minutes**, assistant optional—document prompts used if any.

Success: another engineer skim-confirms ≥70% fidelity or lists corrections quickly.

Phase — 10-ai-assisted-refactoring-engineering

Directory: `10-ai-assisted-refactoring-engineering/`

Source: `10-ai-assisted-refactoring-engineering/01-scope-migrations-compatibility-regression-safety-and-modernization-ownership.md`

Unit 1 — Scope: AI-assisted refactoring — disciplined change at scale

Mindset shift: refactors are **inventory + safety harness** first; completions are last-mile once invariants locked.

Learning outcomes

- **Refactor ownership framing**: behaviour preservation arguments.
- **Migration ownership** (code + data + config) with explicit phases.
- **Compatibility & contract preservation** when public surfaces exist.
- **Technical debt quantification** heuristic (interest vs principal payments).
- **Architecture alignment checks** against target-state sketches.
- **Regression prevention playbook**: widen tests BEFORE mechanical edits where feasible.
- **Dependency cleanup rationale** (death of dead modules, unify duplicate helpers cautiously).
- **Constraint preservation** during mechanical rename waves.
- **Rollback discipline**: bisect narratives, checkpoints, tagging.
- **Large-scale refactoring sequencing**: strata (types → internals → callers) minimise broken intermediate states when possible.
- **Legacy modernization** trade-offs (strangler façade vs big-bang condemnation).
- **Change safety RACI-lite** pairing IC + reviewer expectations.

Linkage: reinforces `05-*`, validates via `07-*`, discovers via `09-*` prior mapping.

Source: `10-ai-assisted-refactoring-engineering/02-labs-compat-first-refactor-migration-matrix-and-rollforward-plan.md`

Unit 2 — Labs: phased refactor tabletop

Identify a refactor surface (duplicate error mapping, sprawling service class god object, transitional DTO bridging).

Produce:

1 — Invariants list (≥8 bullets)

Operational + behavioural + performance guard rails.

2 — Compatibility matrix table

Consumers | breakage risk | mitigation | deprecation window signal

3 — Automated safety net backlog

Existing tests augmented + gaps newly specified.

4 — Mechanical wave plan

Step ordering with **stop gates** (“after wave 2, deploy behind flag X” hypothetical ok).

5 — Abort criteria

Signals to halt assistant-driven sweeping edits immediately.

Conduct **dry rehearsal**: narrate rollback if Wave 3 introduces subtle race.

Deliverable: `REFACTOR-PLAN.md` suitable for attaching to RFC.

Phase — 11-ai-testing-engineering-for-assisted-codebases

Directory: `11-ai-testing-engineering-for-assisted-codebases/`

Source: `11-ai-testing-engineering-for-assisted-codebases/01-scope-pyramids-contract-flakiness-load-and-verification-integration.md`

Unit 1 — Scope: AI testing engineering — amplified velocity demands amplified proof

Mindset shift: generated code increases **derivative change rate** → testing strategy must widen signal, not trivia counts.

Learning outcomes

- **Unit ownership mindset**: deterministic pure seams first.
- **Integration ownership**: realistic boundary mocks but not brittle network chatter.
- **Contract ownership**: schema / OpenAPI-ish parity probes when applicable.
- **Property-ish thinking** optional depth (fuzz light, invariant randomisation).
- **Mutation testing awareness** as blind-spot oracle for sloppy suites.
- **Regression ownership expansions**: snapshot judicious usage anti-patterns called out.
- **Flaky test ownership**: classify environmental vs data race vs time—remove lottery from CI signal.
- **Chaos mindset hooks** (timeouts, partitioned failures simulations) high-level.
- **Load awareness sketches** distinguishing microbench vs systemic soak.
- **Failure reproduction fidelity** stories for regressions surfaced post assistant merge.
- **CI verification ownership**: pipeline stages aligning risk class.
- **Coverage ownership scepticism**: thresholds vs behavioural sufficiency narratives.
- **Verification pipeline cohesion** with `07-*`.

Source: `11-ai-testing-engineering-for-assisted-codebases/02-labs-flake-hunt-contract-probe-and-testing-pyramid-upgrade.md`

Unit 2 — Labs: strengthen the weakest signal chain

Locate **worst flaky / lowest power** automated test touching code you influence.

Activities:

A — RCA mini retro

Enumerate historical failure modes ignoring “just retry CI” pathology.

B — Stabilisation patch idea set

Isolation / fixtures / teardown / faker seeding / parallelism constraints.

C — Supplementary invariant test add

Prefer single high-signal test over noisy shallow additions.

D — Contract check stub

Ideate YAML/JSON parity validation post schema migration.

E — Mutation thought experiment

If one line mutated silently, **which assertion still fires**?

Track before/after **mean CI green expectancy** subjective note.

Phase — 12-ai-professional-practice-and-human-review-discipline

Directory: `12-ai-professional-practice-and-human-review-discipline/`

Source: `12-ai-professional-practice-and-human-review-discipline/01-scope-small-diff-reproducibility-observable-debugging-and-non-magic-rules.md`

Unit 1 — Scope: AI professional practice — rituals beat heroics

Mindset shift: professional AI workflow is boringly **repeatable** — speed emerges from restraint.

Learning outcomes

- **Small diff philosophy**: minimal blast incremental integration.
- **Review ownership sharpened**: reviewer reads like adversary with AI bias awareness.
- **Merge ownership friction stays human**: author explains intent succinctly—not diff dump only.
- **Verification-before-merge invariant** reiterated.
- **Explicit acceptance criteria echoed** tying manager expectations.
- **Architecture preservation instincts** resisting drive-by layering inversions.
- **Deterministic workflow patterns**: scripted commands reproducible teammate→teammate.
- **Reproducibility hooks**: seeded randomness pinned, caches declared.
- **Observability-first debugging**: logs/metrics breadcrumbs before refactor fantasy.
- **Rollback-before-optimisation discipline**: correctness path stable prior micro-perf churn.
- **Migration safety shorthand checklist**.
- **Minimal context principle** tie-back to budgeting `19-*`.
- **No blind generation oath**: generation always paired checkpoint.
- **No magic ownership mantra**: miraculous fix demands exposure of mechanism test.

Source: `12-ai-professional-practice-and-human-review-discipline/02-labs-review-checklist-ai-honest-mini-rfc-merge-story.md`

Unit 2 — Labs: procedural upgrades

Produce `REVIEW-WITH-AI.md` merging:

Top 15 review questions customised to your stacks (Symfony/Go/DevOps overlays welcome).

Mandatory sections:

- Security & privacy red flags.
- Behavioural regressions lurking in “simple” codegen.
- Observable debugging hooks unaffected?
- Architectural boundary violations heuristic list.
- Flake / nondeterminism regression triggers.

Conduct **retro on a merged PR**: identify one moment AI sped correctness, one moment it threatened it—bullet lessons.

Phase — 13-agent-orchestration-and-multi-role-workflows

Directory: `13-agent-orchestration-and-multi-role-workflows/`

Source: `13-agent-orchestration-and-multi-role-workflows/01-scope-planner-executor-reviewer-topology-retries-delegation.md`

Unit 1 — Scope: agent orchestration engineering — choreography with accountability

Mindset shift: multiple semi-autonomous steps require **topology + state machine thinking** — planners, executors, reviewers need explicit delegation contracts.

Learning outcomes

- **Planner pattern**: decomposition referencing existing architecture map (`09-*`).
- **Executor pattern**: bounded autonomy windows + tool constraints.
- **Reviewer / adversarial role**: rejects patch lacking evidence or violating constraints (`07-*`).
- **Multi-agent coordination cautions**: hidden consensus illusions unless externalised diff reviews happen.
- **Delegation ownership clarity**: RACI overlays on autonomy segments.
- **Approval ownership**: enumerated gate transitions.
- **Capability routing**: model vs tool vs human specialist selection heuristics.
- **Retry ownership**: backoff + stop conditions — avoid doom loops (`08-*` echo).
- **Fallback ownership**: degrade plan when tool partial failure (`14-*` preview).
- **Escalation ownership**: stuck detection triggers.
- **Workflow ownership artefacts**: ephemeral plan docs / decision logs versioning.
- **Orchestration anti-patterns**: “telephone game summarisation loses truth”.
- **Agent topology sketches**: sequential vs DAG vs parallel guarded merge.
- **State ownership**: what lives in ephemeral chat vs persisted branch vs ticket.

Cross-link governance `06-*`, tools `14-*`, failures `08-*`.

Source: `13-agent-orchestration-and-multi-role-workflows/02-labs-orchestration-blueprint-plan-reviewer-loop-and-stop-conditions.md`

Unit 2 — Labs: blueprint a guarded multi-step workflow

Produce `ORCHESTRATION-BLUEPRINT.md` for a hypothetical feature (“add read-model projection consumer” flavour optional).

Mandatory elements:

Plan states table: `idle → plan → revise → approve? → exec → verify → merge-hold`

Each transition lists **OWNER** (human / agent role) + **TOOLS allowed** + **STOP condition**.

Embed one **planned failure injection** (“reviewer rejects missing benchmark”) illustrating recovery—not happy path fantasy.

Conduct **mental dry run** ≤15 minutes narrating divergence if executor skipped verification node.

Phase — 14-tool-orchestration-mcp-hooks-and-controlled-automation

Directory: `14-tool-orchestration-mcp-hooks-and-controlled-automation/`

Source: `14-tool-orchestration-mcp-hooks-and-controlled-automation/01-scope-tool-selection-permissions-retries-partials-recovery-verification.md`

Unit 1 — Scope: tool orchestration — reliability of capability surfaces

Mindset shift: MCP / terminal / browser tooling extends reach — expands **failure + permission** frontier.

Learning outcomes

- **Tool routing rationale**: cheapest reliable truth path first (git status vs speculative reasoning).
- **Capability ownership**: which tools permissible per risk envelope (`06-*` alignment).
- **Retry etiquette**: differentiated for idempotent reads vs destructive writes.
- **Fallback ladders**: degraded manual path when tooling flakes.
- **Tool degradation signalling**: timeouts, truncated JSON, stale workspace roots.
- **Partial failure narratives**: half-applied filesystem patch + transactional thinking.
- **Approval boundaries intersecting tooling**: prompts that could exfil secrets must be gated.
- **Permission ownership hooks**: scopes / PAT lifetimes principle-level.
- **Retry escalation thresholds** → human when consecutive autonomous failures accumulate.
- **Recovery ownership**: rewind local partial apply / reclone cautions high-level.
- **Verification tying**: rerun failing command proving fix post tool-assisted patch.

Source: `14-tool-orchestration-mcp-hooks-and-controlled-automation/02-labs-tool-routing-matrix-degrade-modes-incident-micro-runbook.md`

Unit 2 — Labs: degrade-mode micro-runbooks

Produce **THREE** hypothetical tool incidents (timeouts, truncation, accidental wrong workspace root assumed).

Structure each:

Cause hypothesis • User-visible symptom • Safe immediate mitigation • Verification step • When to escalate • Guardrail tweak (RULE / MCP config / onboarding doc)

Compose **routing matrix** table:

Goal | Prefer tool | Fallback | Forbidden path | Verification | Rollback cue

Conduct **paired reflection**: Identify one MCP capability you personally would sandbox first in a new codebase—justify with threat model sentence.

Phase — 15-ai-architecture-reasoning-and-assisted-design-authority

Directory: `15-ai-architecture-reasoning-and-assisted-design-authority/`

Source: `15-ai-architecture-reasoning-and-assisted-design-authority/01-scope-bounded-contexts-tradeoffs-adrs-and-nfr-ownership-under-acceleration.md`

Unit 1 — Scope: AI architecture engineering — assist without surrendering coherence

Mindset shift: architecture remains **negotiated coherence** assisted by drafts — not improvised sprawl gated only by completions.

Learning outcomes

- **Bounded contexts** discipline refresh (overlap with `09-*` mapping).
- **Ownership hierarchy overlays** aligning teams / modules / runbooks / on-call rotations.
- **Tradeoff engineering** articulation sharpened via assistant scenario simulation but human signs.
- **Architecture reasoning artefacts**: concise ADRs, decision logs, phased target diagrams.
- **NFR anchors**: reliability scalability cost operability/security tension quadrants succinct.
- **Specification ownership interplay** bridging runtime checks `16-*`.
- **Reliability / consistency / scalability narratives** resisting buzzword fluff—tie to empirical signals.
- **Migration ownership bridging** modernization arcs.
- **Dependency ownership** resisting hub nodes sneaking inward.
- **Failure modeling overlays** injecting AI operational failure modes.
- **Architecture preservation vigilance**: AI-proposed layering inversions resisted until ADR-level acceptance.

Source: `15-ai-architecture-reasoning-and-assisted-design-authority/02-labs-adr-draft-adversarial-critique-and-revisit-criteria.md`

Unit 2 — Labs: miniature ADR + AI challenge loop

Draft **ADR-PROTOTYPE**: proposed structural change influenced / critiqued by assistant.

Stages:

1 — Human first draft rationale

2 — AI sceptic pass bullets

Enumerate **≥5 risks** intentionally adversarial—rate severity.

3 — Revised ADR merges mitigations / deferred decisions explicitly.

4 — Closure criteria list

measurable signals deciding revisit window.

Conduct **timed defence** (solo voice acceptable): summarise tradeoffs ≤120s without jargon stacking.

Phase — 16-ai-specification-runtime-enforcement-automation-and-drift-hooks

Directory: `16-ai-specification-runtime-enforcement-automation-and-drift-hooks/`

Source: `16-ai-specification-runtime-enforcement-automation-and-drift-hooks/01-scope-executable-constraints-ci-probes-consistency-scanning-and-runtime-drift-signatures.md`

Unit 1 — Scope: AI specification runtime — constraints that travel with execution

Mindset shift: push critical invariants closer to automation that can block drift — bridging static review + operational signals.

Learning outcomes

- **Executable constraints ethos**: lint rules, architectural tests, CI policy snippets.
- **Specification runtime verification** concepts: conformance checks integrating generated code vs schemas.
- **Acceptance probes** scripted post-deploy synthetic transactions (high-level—not vendor-specific playbook).
- **Specification consistency scanners** aligning OpenAPI/proto ↔ implementations (conceptual—you choose stack fit).
- **Runtime drift ownership**: metrics implying silent semantic skew (latency distributions, anomaly counters, staleness gauges).
- **Implementation verification bridging** continuous integration marrying PR artefacts + probes.
- **Repair ownership tightening** once drift surfaced—loops back to `07-*` disciplined repairs.
- **Quality ownership overlays** covering observability contract adherence—not only happy-path logic.
- **Benchmark recurrence** guarding performance regressions on hot paths touched by codegen.
- **Constraint validation layering** alleviating brittle human checklist scaling limits.

Source: `16-ai-specification-runtime-enforcement-automation-and-drift-hooks/02-labs-drift-hooks-backlog-mini-conformance-matrix-and-alert-excerpt.md`

Unit 2 — Labs: conformance & drift signalling backlog

Produce `DRIFT-HOOK-BACKLOG.md` containing:

Table A — Signals

Invariant | Detector idea (lint / test / metric) | Blast if missed | Noise risk | Mitigation |

Table B — Staged rollout waves

Cheap early signals vs heavier integration probes sequencing.

Pick **two** hypothetical AI codegen failure classes (mis-serialisation field drift, subtly widened latency envelope) sketch alert shapes + avoidance of alert fatigue tactic.

Phase — 17-ai-security-engineering-governance-and-supply-chain

Directory: `17-ai-security-engineering-governance-and-supply-chain/`

Source: `17-ai-security-engineering-governance-and-supply-chain/01-scope-prompt-risk-unsafe-diff-patterns-secrets-deps-and-boundaries.md`

Unit 1 — Scope: AI security engineering — threat model for accelerated change

Mindset shift: copilots expand **attack surface velocity** — prompt + tool + dependency fronts.

Learning outcomes

- **Prompt injection awareness**: untrusted content vs trusted instructions; poisoning via pasted logs or issue bodies.
- **Unsafe generation detection heuristics**: secret patterns, credential sprawl, destructive mass operations, privilege escalation templates.
- **Trust boundaries**: where AI suggestions halt pending human escalation.
- **Approval boundaries** tying sensitive infra / billing / PII-touching merges.
- **Dependency trust uplift**: pinning, integrity verification, deprecation of speculative “nice” libraries sourced only from completions.
- **Secret ownership rituals**: forbidding pasted tokens into assistant windows; ephemeral env segregation.
- **Dependency security scanning synergy** connecting review flow + codegen suggestions.
- **Permission ownership**: least-privilege MCP / CI PAT scopes; rotation intuition.
- **Supply chain scepticism**: copy-pastes of unknown snippets flagged before merge.
- **Validation discipline** reinforcing “assume hostile diff until proven benign”.
- **Attack surface enumeration** augmented with automation hooks (`git`, browser, MCP file writes).
- **Safe execution / sandbox instincts** aligning with organisational policy (`06-*`).
- **Context isolation**: dataset separation preventing accidental leakage of staging prod-like fixtures into logs committed for assistance.

Source: `17-ai-security-engineering-governance-and-supply-chain/02-labs-red-team-prompts-mcp-abuse-checklist-and-dep-triage.md`

Unit 2 — Labs: red-team yourself

Tasks:

1 — Craft benign-looking malicious-ish prompt excerpt

Demonstrate hypothetical injection coaxing destructive shell—**do NOT execute**. Document interception guard.

2 — Sensitive diff scavenger hunt

Locate five historical commit anti-patterns (from memory or sanitized stories) completions might reproduce.

3 — MCP / tool threat row

Enumerate three concrete abuse paths + corresponding guard (rate limit, manual approval, alternate read-only tool).

4 — Secret hygiene checklist

≤10 bullets enforceable on team slack / onboarding.

5 — Supply chain quick triage flowchart

Decide accept / fork / reject heuristics for suggested dependency addition.

Deliverable: `SECURITY-AI-CHECKLIST.md` attachable to team handbook appendix.

Phase — 18-ai-evaluation-benchmark-quality-and-return-on-assistance

Directory: `18-ai-evaluation-benchmark-quality-and-return-on-assistance/`

Source: `18-ai-evaluation-benchmark-quality-and-return-on-assistance/01-scope-benchmark-design-golden-sets-metrics-drift-latency-and-cost.md`

Unit 1 — Scope: evaluation engineering — measure to improve, not for vanity dashboards

Mindset shift: disciplined evaluation distinguishes **measurable leverage** from story-driven “feels faster” AI adoption claims.

Learning outcomes

- **Benchmark ownership lifecycle**: hypothesis → curated tasks → rubric → scoring → regression thresholds.
- **Regression benchmark layering** capturing model/tooling/rule updates impact.
- **Golden dataset stewardship**: versioning, diversification, rotten-case pruning.
- **Quality metrics beyond pass/fail**: partial credit rubrics for exploratory tasks (documented subjective transparency).
- **Hallucination / fabrication probes** with objectively checkable citations.
- **Repair metrics**: rework cycles minutes + human interruptions count qualitative.
- **Latency ownership**: end-to-end latency including human comprehension / review—not model-only.
- **Cost ownership**: approximation discipline for assisted workflows (calls / tokens heuristic).
- **Drift vigilance**: quality decay detectors after infra changes.
- **Evaluation governance**: curator role + freshness cadence.
- **Leakage scepticism**: training overlap awareness for leaked internal snippets (conceptual—not conspiracy).
- **Ethical dataset handling**: sanitisation norms for artefacts leaving team boundary.

Source: `18-ai-evaluation-benchmark-quality-and-return-on-assistance/02-labs-golden-microtask-rubric-and-metrics-sanity-comments.md`

Unit 2 — Labs: miniature evaluation capsule

Produce `EVAL-GOLDEN-V0.md` with **≤12 curated micro-tasks** spanning:

syntax fix • subtle logic bug • small refactor preserving API • exploratory architecture question needing refusal guard • security-sensitive stub requiring escalation • flaky test chase • ambiguous spec requiring clarification question

For each capture:

Difficulty tag | Expected success shape | Verification command | Hazard if automated blindly

Conduct **pseudo A/B commentary**: hypothetical “ruleset tweak A vs B”—which KPI moves first?

Reflect: metric most likely lying if taken alone—justify.

Phase — 19-context-cost-latency-and-quality-triangulation

Directory: `19-context-cost-latency-and-quality-triangulation/`

Source: `19-context-cost-latency-and-quality-triangulation/01-scope-budget-retrieval-compression-reuse-tradeoffs-and-metrics.md`

Unit 1 — Scope: context cost engineering — optimise the expensive middle

Mindset shift: shrink context footprints **without starving truth** — triangulating cost, latency, and answer quality consciously.

Learning outcomes

- **Token budget analogue reasoning**: relative weight intuition when exact counts unseen.
- **Context budget pacing cycles**: summarise → anchor → escalate retrieval narrowness progressively.
- **Compression strategy layering**: synopsis + discriminative anchors vs dumping whole files blindly.
- **Retrieval optimisation heuristics**: narrow roots, prune stale lumps, suppress duplicate hotspots.
- **Latency ownership bridging** perceived human waits (assistant tool hops, human re-read churn).
- **Context reuse disciplined**: staleness auditing for saved summaries / SKILL duplicates.
- **Parallelisation allowances** when read-only and conflict-free versus coherence hazards merging parallel branches of reasoning.
- **Cost optimisation instincts**: diminishing returns thresholds—fallback to handcrafted micro-patch.
- **Quality vs cost tradeoffs** surfaced explicitly—not silent shrink.
- **Optimisation anchoring**: pair improvements with KPIs from `18-*` to avoid jittery tweaking.

Source: `19-context-cost-latency-and-quality-triangulation/02-labs-context-diet-journal-and-parity-experiments.md`

Unit 2 — Labs: context diet experiment journal

Maintain `CONTEXT-DIET-JOURNAL.md` over **five** real-sized tasks documenting:

Estimated relative context weight (S/M/L) | Outcome fidelity (pass/partial/fail) | Correction cycles | Surprise omissions | Adjustment applied next time

Identify **≥2** incidents where richer context slowed review without improving correctness—articulate concrete workflow refactor for iteration N+1.

Bonus: correlate one heavy-context failure with remediation already suggested in `03-*` compression capsule pattern.

Phase — 20-ai-in-production-release-ci-incident-and-operational-integration

Directory: `20-ai-in-production-release-ci-incident-and-operational-integration/`

Source: `20-ai-in-production-release-ci-incident-and-operational-integration/01-scope-ci-deploy-review-rollbacks-telemetry-audit-and-incident-hooks.md`

Unit 1 — Scope: production AI systems — when assistance hits release trains

Mindset shift: fold AI-accelerated workflows into **operational reality**: CI, deployments, incidents, audits.

Learning outcomes

- **CI integration posture**: stage ordering hardened when generated code spikes (lint → typecheck → tests → security scanners).
- **Deployment awareness**: migrations + feature-flag pairing for risky completions.
- **Review ownership at scale**: risk-weighted review depth vs exhaustive line-by-line impossibility narrative.
- **Rollback awareness**: reversibility stories for AI-touched releases (flags, forward fixes).
- **Observability integration**: transient error-shape monitoring after sweeping edits.
- **Incident ownership adaptations**: lightweight “AI-assisted change” tagging for triage heuristics (blameless—signal only).
- **Audit artefacts**: approvals + policy alignment evidence when mandated.
- **Operational ownership aligning on-call readiness** vs elevated change velocity.
- **Release window hygiene** articulating freezes + compounded risk acceptance.
- **Production constraints bridging** latency/capacity hotspots touched by completions.
- **Operational verification layering** synthesising probes + dashboards + escalation cues.

Source: `20-ai-in-production-release-ci-incident-and-operational-integration/02-labs-incident-storyboard-assisted-release-deploy-checklists.md`

Unit 2 — Labs: assisted-release tabletop

Fabricate `INCIDENT-STORYBOARD.md` for a fictional release where an AI-guided mass refactor subtly widened DB connection pool concurrency.

Sections:

Timeline • Detection • Confusion vectors • Differentiation diagnostics • Mitigation • Long-term instrumentation • Retro action items (link drift-hooks ideas to `16-*`).

Produce a **paired checklist** for a hypothetical deploy ticket:

Pre-merge • Post-merge 15 m • Post-merge 24 h • Rollback trigger • Communication escalation path

---

Phase — 21-ai-leadership-organisational-models-and-cultural-scaling

Directory: `21-ai-leadership-organisational-models-and-cultural-scaling/`

Source: `21-ai-leadership-organisational-models-and-cultural-scaling/01-scope-governance-policy-standards-adoption-metrics-and-role-modelling.md`

Unit 1 — Scope: AI leadership engineering — organisational leverage with adult supervision

Mindset shift: scale practices through standards, narratives, mentoring—not individual hero prompting alone.

Learning outcomes

- **AI governance escalation** tailoring org risk appetite vs team autonomy (extends `06-*`).
- **Team standards authoring**: RULE / SKILL parity, versioning, deprecation discipline.
- **AI operational policy** communication compressing bureaucracy into workable guardrails.
- **Workflow standardisation** aligning review + verification minimums (`12-*` echo).
- **Architecture influence storytelling** winning adoption without authoritarian decree fatigue.
- **Review culture uplift** emphasising sceptical empathy toward assisted diffs.
- **AI adoption ownership metrics** avoiding vanity counts—pair with `18-*`.
- **Organisational ownership** bridging platform + product + compliance stakeholders.
- **Risk ownership narratives** articulate residual vs mitigated crisply for exec briefings.
- **AI operating model** variants (embedded champion vs centre-of-excellence hybrids) strengths/weaknesses.
- **Engineering leadership signalling**: psychological safety admitting model mistakes publicly for learning multiplier.

Source: `21-ai-leadership-organisational-models-and-cultural-scaling/02-labs-standards-one-pager-risk-register-rollout-mini-plan.md`

Unit 2 — Labs: leadership artefact sprint

Produce `AI-STANDARDS-ONEPAGER.md` you could socialise broadly:

Audience split: IC • EM • Principal • Compliance-friendly bullet

Include:

Rolling review minimums • Secret prohibitions snippet • MCP permission baseline • Incident disclosure tone • Adoption metric sanity guard • Mentorship pairing pattern for juniors + AI responsibly

Conduct **risk register mini-table** listing top five adoption risks mitigations owners review cadence.

Phase — 22-principal-capstone-integration-labs-pressure-scenarios-and-defence

Directory: `22-principal-capstone-integration-labs-pressure-scenarios-and-defence/`

Source: `22-principal-capstone-integration-labs-pressure-scenarios-and-defence/01-scope-cross-cutting-modernization-tradeoff-incident-architecture-integration.md`

Unit 1 — Scope: Principal Cursor capstone labs — integrated pressure arcs

Mindset stitch: weld earlier units into coherent **principal-level** rehearsals—architecture, modernization, resilience, storytelling.

Capability activation map (pick 2 deep dives + 3 supporting threads)

Modernization arcs (discovery `09-*`, refactor `10-*`, migrations, governance `06-*`) • Architecture facilitation & sceptical facilitation `15-*` • Incident facilitation post AI-generated change hypotheses • Evaluation & cost triangulation (`18-*`, `19-*`) • Production integration `20-*` • Leadership artefacts `21-*` • Verification / failure ladders `07-*` / `08-*` as safety spine.

Articulate explicitly which modules you consciously **defer**—principal honesty > pretend omniscience.

Source: `22-principal-capstone-integration-labs-pressure-scenarios-and-defence/02-labs-modernization-review-incident-narratives-and-tradeoff-defense-sprints.md`

Unit 2 — Labs: staged pressure arcs (solo or pair)

Complete **THREE** artefacts (distinct):

Artefact X — Dual-depth modernization storyline

Technical chain (expand/contract, compatibility, rollback) + sponsor summary ≤120 words + principal risks explicit.

Artefact Y — Post-incident review outline

Facts timeline • Corrections hypotheses • Measurement gaps • Ownership assignments • Automation follow-ups tying `16-*`.

Artefact Z — Defence drill

Timed spoken or written stance defending **conscious trade rejection** (“we slowed AI velocity here intentionally because …”) anchored to KPI + risk—not ideology.

Retro: list **five** lingering blind spots deserving future deepening outside this Cursor path corpus.
